Notes on Combinatorial and Probabilistic Algorithms

Riccardo Lo Iacono

July 22, 2026

Contents.

– 1 – The analysis of Algorithms.

As a good computer scientist knows, not all algorithms are the same nor are the resouces these use; it is for these reasons that we need ways to analyze algorithms. In what follows, we first recall what asymptotic notation is, and then introduce the notion of *amortized analysis*.

– 1.1 – Asymptotic notation.

That idea behind asymptotic notation is that of analyzing algorithms, with respect to a given resource, more often then not being time, without caring about constants.

We recall the existence of three major notation: the *omega* notation (Ω), the *theta* notation (Θ) and the big-o notation ($\mathcal{O}$). Let $g(n)$ and $f(n)$ be two functions, then we define each notation as follow:

$$\begin{aligned}
\Omega(f(n)) =& \{g(n) \mid \exists c > 0, n_0 \geq 0 \text{ s.t. } g(n) \geq cf(n), \forall n \geq n_0\} \\
\Theta(f(n)) =& \{g(n) \mid \exists c_1, c_2 > 0, n_0 \geq 0 \text{ s.t. } c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\} \\
\mathcal{O}(f(n)) =& \{g(n) \mid \exists c, n_0 \geq 0 \text{ s.t. } g(n) \leq cf(n), \forall n \geq n_0\}
\end{aligned}$$

Although we give the definition in set notation, we write $g(n) = \mathcal{O}(f(n))$ to mean $g(n) \in \mathcal{O}(f(n))$; we do similarly for the other notations. Additionally, unless stated otherwise, assume we are interested in the time complexity of the algorithm.

– 1.2 – Amortized analysis.

Let us consider the following: assume we are given a data structure, and we are asked to compute a sequence of m operations on it; what is the overall complexity of all m operations, i.e, what is the time needed to compute all m operations.

It follows easily that, applying a worst case analysis, we need $m \cdot T_{MAX}(n)$, where $T_{MAX}(n)$ is the time complexity of the most expesive operation. For instance, under the worst case assumption, a sequence of m deletion from a tree has a complexity of $\mathcal{O}(m \cdot n)$.

We now introduce the notion of *amortized analysis*, and show how, over a sequence of operations, the real time complexity of an algorithm differs from the worst case one.

In its essence, amortized analysis does nothing more than averaging the complexity of all operations. That is, if $T_1(n)$ is the complexity of the first operation, $T_2(n)$ the one for the second operation, ..., $T_m(n)$ the complexity

of the m-th operation, then the amortized time is given as

$$T_{amortized}(n) = \frac{1}{n}\sum_{i=1}^{m} T_i(n).$$

In other words, amortized time represents the time needed for each of the m operations.

The next section describes a simple ways to perform amortized analysis.

– 1.2.1 – Accounting method.

The core idea of the method follows that used by banks. In more details: to each operation we assign a cost, which represents its *amortized cost*. When the amortized cost of an operation exeeds its real cost, we are left with some credit that we can use to help pay other operations.

Example: Consider we need an algorithm to increment a n bits counter by an amount m. From a worst case analysis perspective, the algorithm needs a quadratic time, as the n-th bit may need to be flipped m times. It comes with no surprise that such a cost is well above the real one, as we are about to see.

Let us proceed this way: for each bit flip, assume that we are asked to pay a coin; then we proceed by paying two coins when flipping a bit to 1. The cost of the whole algorithm is easily seen to be given by the number of total bit flipped. But the following can be observed: the first increment, leaves us with a coin of credit, the second one does the same, etc. Thus, each operation requires a constant amount; that is, the amortized cost of the whole algorithm is $\mathcal{O}(n)$.

Remark: Other methods to perform amortized analysis exist, and their use varies with the algorithm we need to analyze. Though, all follow the same core idea: we overpay for some operations and use the credit thus produced to pay for others.

– 2 – The dictionary problem.

Now, we focus our attention to one of the most common problems in computer science, namely, the *dictionary problem* which can be roughfully defined as follow: given a set of items S, we want to define a data structure such that the following three operations are performed efficiently.

Search(S, x)
Insert(S, x)
Delete(S, x)

The following assumpion are considered: accessing or deleting an item i takes time i, insertion takes time $i + 1$ and puts the element at the end of the list.

We begin our discussion on the problem with the following trivial remark: if the elements have some kind of total order relation, then trees provide the best approach; if not the best possible solution is given by lists.

Here we discuss two type of solutions, namely, self-adjusting lists (for the case in which no order relation exists) and self-adjusting trees (for the other case). Specifically, for self-adjusting trees we discuss, in order, red-black trees (**??**) and splay trees (**??**).

– 2.1 – Self-adjusting lists.

Our discussion on self-adjusting lists will proceed as follow: first, we introduce the definition of self-adjusting list; then, we focus our attention on some of the most common rules used to keep a list organized.

Conceptually, self-adjusting lists are trivial: these are lists in which, after accessing or inserting an item, some update procedure is called to keep the list organized. These procedure are defined according to some rules, among which those that stands out are

- *Frequency Count (FC)*: we mantain an array that stores at each position i, the count of how many times item i has been accessed or added;
- *Move-To-Front (MTF):* each time an item is added, it is moved to the head of the list, while each accessed item is moved slighly closer to the root;
- *Transpose (T):* accessing or inserting an item, implies an exchange of the added/accessed item with the one on its left.

Let $\mathbb{E}_A(p)$ be the expected cost to access an item using algorithm A, where $p = \{p_1, p_2, \ldots, p_n\}$ is the probability distribution associated to the sequence of operations. Assume DP to be an algorithm that orders the elements according to p. By the law of large numbers it follows $\mathbb{E}_{FC}(p)/\mathbb{E}_{DP}(p) = 1$. Additionally,

one could prove that $\mathbb{E}_{MTF}(p) \leq 2\mathbb{E}_{DP}(p)$, and as proved in [1] $\mathbb{E}_{T}(p) \leq \mathbb{E}_{MTF}(p)$. A similar result can be easily derived for insertions and deletions. From the above, it follows that, at least in theory, both FC and transpose out perform MTF. As we are about to show, theory and practice not always coincide. What we want to show is that, although in theory FC and transpose are better then MTF, in practical applications MTF performs better.

For any algorithm A, let us denote with $C_A(s)$ the total cost of all operations, by $F_A(s)$ the number of exchanges that move an item to the head of the list, and by $P_A(s)$ any other exchange. The following holds.

Theorem 2.1 ([2, Theorem 1.]): *For any Algorithm A and any sequence of operations s starting with the empty set*

$$C_{MTF}(s) \leq C_A(s) + P_A(s) - F_A(s) - m.$$

Theorem 2.1 and its generalized version ([2, Theorem 2.]), show that MTF differs by any other algorithm by a most a factor of 2. No analogous result holds for FC and transpose.

References.

[1] R. L. Rivest. “On Self-Organizing Sequential Search Heuristics”. In: *Commun. ACM* 19.2 (1976), pp. 63–67. DOI: 10.1145/359997.360000.

[2] D. D. Sleator and R. E. Tarjan. “Amortized Efficiency of List Update and Paging Rules”. In: *Commun. ACM* 28.2 (1985), pp. 202–208. DOI: 10.1145/2786.2793.